Experiment - 1

Aim: Study of various network commands used in linux and windows

Basic Networking commands:

(i) arp - a : ARP is short form of address resolution protocol. It will show IP address of your computer along with IP address and MAC address of your router

(ii) hostname: Simplest of all TCP/IP commands. Simply displays name of computer.

(iii) ipconfig /all : This command displays detailed configuration information about your TCP/IP connection including route, gateway, DNS, DHCP, and type of ethernet adapter in your system.

(iv) nbtstat - a : Helps solve problems with NetBIOS name resolution. (Nbt stands for NetBIOS over TCP/IP)

(v) netstat : (network statistics) netstat displays variety of statistics about computers active TCP/IP connections. It is command line tool for monitoring network connections both incoming & outgoing as well as viewing routing tables, interface statistics etc. eg: - netstat - r

(vi) nslookup : ('name server lookup) is a tool used to perform DNS lookups in linux. It is used to display DNS details, such as IP address of particular computer, MX records for domain or NS servers of domain. nslookup can operate in 2 modes: interactive & non-interactive. eg: - nslookup www.google.com

(vii) pathping : unique to window's & is basically a combination of ping & Tracert commands. Pathping traces rout to destination address then lanches 25 second test of each router along way, gathering statistics on rate of data loss along each hop.

(viii) ping : Best way to test connectivity btw 2 nodes. uses ICMP to communicate to other devices.

(ix) Route : used to show/manipulate IP routing table. primarily used to setup static routes to specific host or network via interface.

Some Important Linux networking commands

(i) ip command - used to view & configure IP addresses, network interfaces, routing tables, other network settings.

Syntax: ip <OPTIONS> <OBJECT> <COMMAND>

Common uses: Display ip address: ip address show

Delete an IP address: ip address del 192.168.1.254/24 dev enp0s3

(ii) ifconfig: used to configure & troubleshoot network interface

(iii) mtr - matt's Traceroute: network diagnostic tool that combines functions of ping & traceroute.

Syntax: mtr <options> hostname/IP

Examples: Basic mtr: mtr google.com

Display numeric IP address: mtr -g google.com

(iv) tcpdump: used to capture & examine network packets travelling through network interface.

It can be installed using: dnf install -y tcpdump

List available interface: tcpdump -D

(v) ping: used to check whether another computer or network device is reachable through an IP address.

Example: ping google.com

Common ping Errors: Destination Host Unreachable

Request time out

Unknown Host

Configuring Ethernet Using nmcli

(i) nmcli is a command-line utility used to manage network connection through Network Manager

(i) Display connection profile : nmcli connection show

(ii) Create new Ethernet connection : nmcli connection add con-name

(iii) Display settings : nmcli connection show

Verification & Troubleshooting :

(i) Display IP address : ip address show enp1s0

(ii) Display routing information : ip route show

(iii) Test connectivity : ping <hostname or IP>

Basic Network Troubleshooting :

1. check whether network cable is properly connected
2. check whether switch and intuface are working
3. check IP address & gateway configuration.

17/9/26

8/10

Experiment - 2

Aim: Study of different types of Network cables

(a) Understand different types of network cables:

(i) Unshielded Twisted Pair (UTP) cable: consist of twisted pairs of copper wires without any protective shielding. cheap & easy to install

(ii) shielded Twisted Pair (STP) cable: similar to UTP, but it has additional protective shield around twisted wires. shield reduces electromagnetic interface & noises.

(iii) Screened Twisted pair (SSTP): provides additional shielding compared with ordinary twisted-pair cables. It offers good protection against electromagnetic interface.

(iv) Coaxial cable: contains central conductor, insulator, sheilding & outer protective layer. provides good protection against interface.

(v) Fibre optic cable: transmits data using light signals through thin glass or plastic fibres. provides very high speed & bandwidth.

(b) Making Ethernet cable:

Commonly made using CAT5/CAT5e or higher category cables & RJ45 connections.

2 common types:

(i) straight-through cable: uses same wiring standard at both ends

(ii) cross-over cable - uses different wiring standards at each ends.

Tools and Materials required:

↳ Ethernet cable

↳ 2 RJ45 connectors

↳ crimping tool

↳ cable stripper / cutter

↳ optional RJ45 connectors boots / shield

↳ cable tester

Steps to Make an Ethernet cable.

(i) prepare cable

(ii) Remove outer cover

(iii) Untwist and arrange wires

(iv) straighten and cut wires

(v) Insert wires into RJ45 connectors

(vi) crimp the connector

(vii) Repeat for other end

(viii) Test the cable

17/9/26

8/10

Experiment - 3

Aim: To study packet tracer tool Installation & User Interface Overview

(c) To understand environment of CISCO PACKET TRACER to design simple network

Introduction:

Network simulation & modelling tool used to design, configure, troubleshoot computer networks without requiring real networking hardware.

Uses of packet tracer:

(i) Allows users to design and simulate compter network without physical equipment.

(ii) Helps students practise networking configuation & troubleshooting

Installation of packet tracker:

To download Packet Tracer, visit Cisco Networking Academy (NetAcad), log in using cisco Networking Academy credentials, select packet tracer, and download suitable for OS.

Windows Installation:

1. Download Packet Tracer setup file
2. Open setup file
3. Accept license agreement
4. Select install location
5. complete installation.

Packet Tracer User Interface

(i) Menu Bar

(ii) Main Toolbar

(iii) Logical / Physical Workspace Tabs

(iv) Workspace

(v) Common Tool bar

(vi) RealTime / Simulation Tool

(vii) Network Component Box

(viii) User-created packet Box

Network simulation using packet tracker:

Step 1 : place network devices

Step 2 : Connect the devices

Step 3 : Configure IP Address

Step 4 : Create a PDU

Step 5 : Observe packet movement

Observation :

Hub : Forwards incoming data to all connected ports except ports from which data was received.

Switch : Forwards data mainly to correct destination port by using MAC address of destination device.

Result :

Cisco Packet Tracer was successfully studied & used to create simple networks using Hub & switch. PDU communication btw PCs was observed.

8/9

9/10

Experiment - 4

Aim: To setup and configure LAN (Local Network Area Network) using switch & Ethernet cables in networking laboratory.

LAN: Network that connects computers & other devices within limited geographical area. LAN allows connected devices to:

↳ share data & files

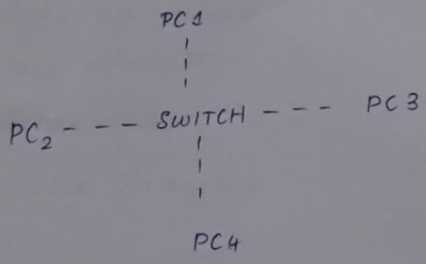
↳ share printers & other resources

Requirements:
- 4 computers (PCs)
- 1 Network switch (8/16/24 ports)
- 4 Ethernet cables
- Network interface cards (NICs)
- Operating system with network configuration support

Procedure to setup LAN:

Step 1: Plan the network

Design suitable network topology according to number and location of computers. Star Topology is used, where all PC's are connected to central switch.



Step 2: Connect devices: Connect each computer to switch using Ethernet cable.

↳ connect PC1 to switch

↳ connect PC2 to switch

↳ connect PC3 to switch

↳ connect PC4 to switch

check that link indicators are active

Step 3: Assign IP Address: Each computer must have unique IP address in same network.

Go to: Network Settings → Ethernet → properties → Internet protocol (TCP/IPv4) → properties.

Select the following IP Address & enter required values

Devices	IP Address	Subnet Mask
PC1	10.1.1.1	255.0.0.0
PC2	10.1.1.2	255.0.0.0
PC3	10.1.1.3	255.0.0.0
PC4	10.1.1.4	255.0.0.0
Switch	10.1.1.5	255.0.0.0

configure Switch :

(i) connect computer to switch using Ethernet cable.

(ii) open web browser and enter switch's management IP address.

(iii) Log in using required username and password.

(iv) Open network / basic settings

(v) Assign switch IP address.

(vi) Save configuration.

Test Network connectivity :

ping command is used to check whether devices can communicate with each other. open command prompt and enter : ping 10.1.1.2 from PC1 to test communication with PC2. If computer are properly configured, reply message will be received.

Observation : The following should be verified.

↳ Ethernet connections are active

↳ Each device has unique IP address

↳ All PCs use same subnet mask

Result : A LAN was successfully set up & configured using 4 computers, network switch, & Ethernet cables.

1/8/9 9/10

Experiment - 5

Packet capture using wireshark

Aim: To capture, filter, view and analyze network packets using wireshark packet sniffer.

Introduction:

↳ wireshark is a network analysis tool used to capture network packets and display their contents in a human readable format.

↳ A packet sniffer captures messages sent and received by a computer and displays the different protocol fields present in those message

↳ wireshark is a passive tool, which means it does not send packets by itself. It captures packets from network interface.

Features of wireshark:

↳ Captures network traffic.

↳ Displays packet details.

↳ Decodes different network protocols.

↳ Provides capture and display filters.

Packet Sniffer structure:

1. Packet List Pane - Displays all captured packets. Each line represents one packet.

2. Packet Details Pane - Displays the protocols and fields of selected packets in an expandable format.

3. Packet Bytes Pane - Displays the selected packet data in hexadecimal format.

Capturing Packets:

(i) Open wireshark

(ii) select local area connection

(iii) Open capture option

(iv) set capture to stop after 100 packets

(v) click start capture

(vi) Packets will appear on screen in real time

(vii) Stop capture after 100 packets

(viii) Save captured packets

Packet Filtering

tcp - Displays TCP packets

udp - Displays UDP packets

arp - Displays ARP packets

dns - Displays DNS packets

Packet capture and analysis

1) TCP/UDP packets

Procedure : (i) Select Ethernet and start capture

(ii) Stop after 100 packets and apply tcp or udp filter

(iii) Inspect packets and view flow graph

(iv) Save captured packets.

2) ARP packets

Procedure : (i) Start packet capture and capture upto 100 packets

(ii) Enter arp in filter box

(iii) Inspect displayed ARP packets and their details

(iv) Save captured packets.

Result : Thus, network packets were successfully captured, filtered, viewed and analyzed using wireshark packet sniffer.

28/7 20/10

Experiment - 6

Aim: To implement error detection and correction using hamming code and verify error correction feature.

Hamming code: error-correction technique used to detect and correct errors in transmitted data. It adds redundant bits to original data

Sender program:

```
string = input ("Enter str: ")
s = ''.join (format (ord (i), '08b') for i in str)
r = 0
while 2 ** r < len (s) + r + 1:
    r += 1
m = len(s) + r
pos = [2 ** i for i in range (r)]
s = s [::-1]
l = [ ]
c = 0
for i in range (1, m+1):
    if i in pos:
        l. append ('p')
    else:
        l. append (int (s[c]))
        c += 1
print ("Parity position: ", pos)
print ("Before parity: ", l)
for p in pos:
    count = 0
    for i in range (p-1, m, 2 * p):
        count += l [i : i+p]. count (1)
    l[p-1] = count % 2
print ("Hamming code: ", l [::-1])
```

Receiver program :

```
f = open ("code.txt", "r")
x = f.readline()
print ("E code: ", x)

e = open ("Original.txt", "r")
y = e.readline()
print ("O code: ", y)

l = x.strip ("[]\n").split(",")

rb = 0
for i in range (len(l)):
    if 2**i >= len(l)+i+1:
        rb = i
        break

pos[]
for i in range (rb):
    pos.append (2**i)
print ("POSITION: ", pos)

q = []
for i in l[::-1]:
    if i.strip() == "1":
        q.append(1)
    else:
        q.append(0)

c = []
for p in pos:
    count = 0
    i = p-1
    while i < len(l):
        count += q[i:i+p].count(1)
        i += 2*p
    if count %2 == 0:
        c.append(0)
    else:
        c.append(1)
print ("BINARY: ", c[::-1])

change = 0
for i in range (len(c)):
    change += c[i]*(2**i)
print ("ERROR: ", change)

q = q[::-1]
if change > 0:
    q[change-1] = 0 if q[change-1]
        == 1 else 1
print ("corrected code: ", q)
```

INPUT: HELLO

OUTPUT:

ERROR: 0

CORRECTED CODE:

[same hamming code]

Result : Thus, hamming code implemented successfully.

Experiment - 7

Sliding window Protocol

Aim: To implement flow control at data link layer using sliding window protocol and simulate flow of frames btw sender and receiver.

Sender program:

```
w = int(input("window size: "))
text = input("Enter text: ")
frames = []
for i in range(len(text)):
    frames.append([i+1, text[i]])
with open("sender_buffer.txt", "w") as f:
    for frames in frames:
        f.write(str(frame[0]) + " " + frame[1] + "\n")
i = 0
while i < len(frames):
    print("\n Sending: ")
    for j in range(i, min(i+w, len(frames))):
        print(frames[j])
try:
    with open("Receiver_buffer.txt", "r") as f:
        ack = int(f.read())
except:
    ack = 0
print("ACK: ", ack)
if ack >= i+1:
    i = ack
else:
    print("Resending...")
print("All frames sent.")
```

Receiver program:

```
with open("Sender_Buffer.txt", "r") as f:
    frames = f.readlines()
expected = 1
for frame in frames:
    no, data = frame.split()
    print("Received: ", no, data)
    if (int(no) == expected:
        print("Correct frame")
        ack = expected
        expected += 1
    else:
        print("wrong Frame")
        ack = 0
    with open("Receiver_Buffer.txt", "w") as f:
        f.write(str(ack))
    print("ACK: ", ack)
print("Receiving completed.")
```

INPUT: HELLO

OUTPUT: Frames received successfully & ACK sent for each frame

Result: Thus, sliding window protocol was implemented successfully

Experiment - 8

Aim: To discover live / active hosts using Nmap with ARP, ICMP, TCP, UDP Scans on TryHackMe.

Introduction to NMAP

↳ Nmap - Network Mapper

↳ free and open-source network scanning tool

↳ used to find live hosts and running services

Nmap suites

1) Nmap Live Host Discovery
2) Nmap Basic port scans
3) Nmap advanced port scans

Subnetworks : subnet is a logical network with its own IP range

Common subnets

Subnet	mask	Approx. Hosts
/16	225.225.0.0	65,000

Enumerating Targets :

1) List : specific IPs / hostname
2) Range
3) Subnet

Discovering live hosts :

Protocol	Layer	Purpose
ARP	Link	Finds MAC address
ICMP	Network	Uses Ping
TCP	Transport	Uses TCP packets

ARP HOST DISCOVERY :

↳ ARP means address resolution protocol

↳ ARP finds MAC address of device

↳ nmap -PR -sn TARGETS

ICMP HOST DISCOVERY

-PE ICMP Echo

-PP ICMP Timestamp

-PM ICMP address mask

TCP SYN PING

↳ Sends TCP SYN packet

↳ Usually uses port 80.

↳ nmap -PS80 TARGETS

TCP ACK KING

↳ sends a TCP packet with ACK flag

↳ Requires privileged account

↳ nmap -PA80 TARGETS

UDP PING

↳ sends UDP packet to target

↳ A closed UDP port may return an ICMP port unreachable message.

REVERSE DNS LOOKUP:

-n Do not perform DNS lookup

-R performs reverse-DNS lookup for all host

Result : Thus, live hosts were successfully implemented.

9/10

Experiment - 9

Implementation of subnetting using cisco packet tracker

Aim : To implement classless IP subnetting using cisco packet tracer.

- Open cisco packet tracer
- create a new network
- Add routers, switches and PCs
- connect devices using suitable cables

Subnetting :

192.168.1.0/24

using /27 gives : 8 subnet

30 usable host addresses per subnet

Configure router :

Open Router → CLI

enable

configure terminal

interface FastEthernet 0/0

ip address 192.168.1.1 255.255.255.224

no shutdown

exit

For another interface

interface FastEthernet 0/1

ip address 192.168.2.1 255.255.255.224

no shutdown

exit

Gigabit Ethernet

enable

configure terminal

interface GigabitEthernet 0/0

ip address 192.168.1.1 255.255.255.224

no shutdown

exit

Configure switch

Open Switch → CLI

enable

configure terminal

interface FastEthernet 0/1

switchport mode access

exit

interface FastEthernet 0/2

switchport mode access

exit

Configure PC's

Open : PC → config → IP configuration

Enter : IP Address

Subnet Mask : 255.255.255.224

Default gateway : Router's IP address

Test the network

1) Open PC → Command prompt

2) Using ping cmd

3) ping another PC to router

Ex : ping 192.168.1.1

If replies are received, network is working correctly

Result : Thus, classless IP subneting was successfully implemented.

Experiment - 10

Internetworking with router using CISCO packet tracer

Aim: To design and configure simple internetwork using router and test connectivity btw 2 PCs.

Network: 1 Router
2 PCs
copper straight-through cables

Procedure: Step 1: Configure router

Open Router → CLI & enter

```
enable
configure terminal
interface FastEthernet 0/0
ip address 192.168.10.1 225.255.255.0
no shutdown
exit

interface FastEthernet 0/1
ip address 192.168.20.1 255.255.255.0
no shutdown
exit
```

Configure PCs

PC → Desktop → IP Configuration

Device	IP Address	Subnet Mask	gateway
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1

Connect PCs to router

↳ Connect PC0 FastEthernet 0 → Router FastEthernet 0/0

↳ Connect PC1 FastEthernet 0 → Router FastEthernet 0/1

↳ Use copper straight-Through cable

Test connectivity

1) Select PC0

2) Use add simple PDU.

3) Click PC1

4) Check whether PDU is successfully delivered.

5) An acknowledge should be received from PC1 to PC0.

Result : Thus, simple internetwork using router was successfully designed and configured.

8/8 x 9/10

Experiment - 11

Static Routing using cisco packet tracer

Aim: To configure and verify static routing using cisco packet tracer

Static Routing:

↳ static routes are manually added to router's routing tables

↳ use ip route cmd to add static route

↳ AD (Administrative distance) decides preferred route

↳ Lower AD = main preferred route

↳ Higher AD = backup route

Networks:

Router	Directly connected network	other network
router0	10.0.0.0, 20.0.0.0, 40.0.0.0	30.0.0.0, 50.0.0.0
router1	20.0.0.0, 30.0.0.0, 50.0.0.0	10.0.0.0, 40.0.0.0
router2	40.0.0.0, 50.0.0.0	10.0.0.0, 20.0.0.0, 30.0.0.0

Router 0 Configuration:

```
enable
configure terminal

ip route 30.0.0.0 255.0.0.0 20.0.0.2 10
ip route 30.0.0.0 255.0.0.0 40.0.0.2 20

ip route 30.0.0.100 255.255.255 40.0.0.2 10
ip route 30.0.0.100 255.255.255 20.0.0.2 20

ip route 50.0.0.0 255.0.0.0 40.0.0.2 10
ip route 50.0.0.0 255.0.0.0 20.0.0.2 20

exit
show ip route static
```

Main routes: 30.0.0.0 → via 20.0.0.2, AD 10

30.0.0.100 → via 40.0.0.2, AD 10

50.0.0.0 → via 20.0.0.2, AD 10

Router 1 configuration :

enable

configure terminal

ip route 10.0.0.0 255.0.0.0 20.0.0.1 10

ip route 10.0.0.0 255.0.0.0 50.0.0.1 20

ip route 40.0.0.0 255.0.0.0 20.0.0.2 10

ip route 40.0.0.0 255.0.0.0 50.0.0.1 20

exit

show ip route static

Verification : check routing table : show ip route static

Test connectivity from a PC : ping <destination-IP>

Trace the path : tracert <destination-IP>

Test backup route : (i) first, check main route

(ii) Delete link btw router 0 and router 1

(iii) Test again using ping/tracert

Delete a static route : First check router : show ip route static

Then remove required route :

no ip route <network> <subnet-mask>

<next-hop>

Result : Static routes were successfully configured verified in cisco packet Tracer.

Experiment

Implement Echo client - server using TCP sockets

Program:

Server code:

```
import socket
server = socket.socket()
server.bind(("127.0.0.1",5000))
server.listen(1)
print("server waiting...")
client, address = server.accept()
print("client connected")
while True:
    msg = client.recv(1024).decode()
    if msg.lower() == "quit":
        break
    print("client: ", msg)
    client.send(("server received: " + msg).encode()
    client.close()
    server.close()
```

client code:

```
import socket
client = socket.socket()
client.connect(("127.0.0.1", 5000))
print("Connected to server")
while ~~msg~~ True:
    msg = input("Enter message: ")
    client.send(msg.encode())
    if msg.lower() == "quit":
        break
    response = client.recv(1024).decode()
    print("Server: ", response)
client.close()
```

INPUT :

Enter message : Hello

Enter message : How are you ?

Enter message : quit

OUTPUT :

Server : Server waiting...
client connected
client : Hello
client : How are you ?

client : Connected to server
Server : Server received : Hello
Server : Server received How are you ?

Result : Thus, tcp echo client - server was successfully implemented and tested.

Experiment 13

Implement your own ping program

Aim: To implement a simple ping program using python

Algorithm: (i) import required modules and define ping() function

(ii) check os and select -n for windows or -c for linux

(iii) create and execute ping cmd using subprocess

(iv) Display ping result or error message

Program:

```
import os
import platform
import subprocess
def ping(host = "google.com", count = 4):
    if platform.system().lower() == "windows":
        param = "-n"
    else:
        param = "-c"
    command = ["ping", param, str(count), host]
    try:
        output = subprocess.check_output(
            cmd,
            universal_newlines = True
        )
        print(output)
    except Exception as e:
        print("ping failed:", e)
if __name__ == "__main__":
    ping("8.8.8.8")
```

INPUT : Host : 8.8.8.8
count : 4

OUTPUT : pinging 8.8.8.8 : bytes = 32 time = 20ms TTL = 117
Reply from 8.8.8.8 : bytes = 32 time = 21ms TTL = 117

Result : Thus, ping program was implemented successfully.

8/8 x 10/10

Experiment - 14

Raw socket packet sniffing

Aim: To write python program using RAW sockets to capture and display network packets.

Algorithm: (i) create raw socket using socket module
(ii) Bind socket to network interface
(iii) receive packets continously using recvfrom() fn
(iv) Display captured packet info.

Program:

```
import socket
S= socket.socket(
    Socket.AF_INET,
    socket.sock_RAW,
    socket.IPPROTO_IP
)
host = socket.getthehostbyname(socket.getthehostname())
S.bind((host,0))
S.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
print("Packet Sniffer started...")
while True:
    packet, address = S.recvfrom(65535)
    print("Packet received from:", address)
    print("Packet data:", packet)
```

INPUT: no direct input is required. program captures packets from network.

OUTPUT: packet sniffer started...

Result: Thus, packet sniffing program using RAW sockets was successfully implemented.

Experiment - 15

web log analysis using webalizer

Aim : To analyze different types of web logic using webalizer tool.

Procedure : (i) Run windows version of webalizer

(ii) select / input web log file downloaded from web

(iii) click run webalizer

(iv) view genuated monthly statistics & host info

output : Monthly statistic

Host

Result : Thus, web logs were successfully implemented.

8/8 X 10/10